baseline_clustering.ipynb

All Cups 3520: baseline кластеризации sparse-данных

Практика — 35 минут: 25 минут на рабочий baseline и 10 минут на одно изменение и обсуждение. Цель — получить воспроизводимые метки кластеров и валидный `baseline_submission.csv`. Скрытой разметки у нас нет: silhouette и устойчивость служат диагностикой, а настоящий Adjusted Rand Score вычисляет All Cups.

Контракт задачи

- Один объект — одна строка sparse-матрицы.
- 77 888 признаков нельзя переводить в dense.
- Нужно выдать целочисленный `TARGET` для каждого `ID`.
- Число кластеров неизвестно: `k` — явная гипотеза, а не найденная истина.
- Метрика платформы инвариантна к перестановке номеров кластеров.

```
from pathlib import Path
import numpy as np
import pandas as pd
from scipy import sparse

from sklearn.cluster import MiniBatchKMeans
from sklearn.decomposition import TruncatedSVD
from sklearn.metrics import adjusted_rand_score, silhouette_score
from sklearn.preprocessing import Normalizer

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

def locate_data_dir():
    candidates = [
        Path("data"),
        Path("competition/data"),
        Path("lesson_05_unsupervised_learning/competition/data"),
    ]
    for candidate in candidates:
        if (candidate / "train.npz").exists():
            return candidate
    raise FileNotFoundError("Run the notebook from competition/ or the repository root")

DATA_DIR = locate_data_dir()
print("Data directory:", DATA_DIR)
```

1. Аудитируем разреженную матрицу

Плотность и число ненулевых признаков на строку влияют на выбор расстояния и алгоритма. Полная dense-копия потребовала бы несколько гигабайт памяти.

```
X_sparse = sparse.load_npz(DATA_DIR / "train.npz").tocsr()
sample = pd.read_csv(DATA_DIR / "sample_submission.csv")

row_nnz = X_sparse.getnnz(axis=1)
density = X_sparse.nnz / (X_sparse.shape[0] * X_sparse.shape[1])
row_norm = np.sqrt(X_sparse.multiply(X_sparse).sum(axis=1)).A1

print("matrix:", X_sparse.shape, X_sparse.dtype, X_sparse.getformat())
print("nnz:", X_sparse.nnz, "density:", f"{density:.4%}")
print("row nnz quantiles:", np.quantile(row_nnz, [0, 0.25, 0.5, 0.75, 1]))
print("row L2 norm range:", row_norm.min(), row_norm.max())
print("sample:", sample.shape, sample.columns.tolist())
```

2. Снижаем размерность без densify

Обычный PCA центрирует признаки и разрушает sparse-представление. TruncatedSVD работает напрямую с разреженной матрицей. После SVD снова нормируем строки, чтобы K-means сравнивал направление профиля, а не только его длину.

```
N_COMPONENTS = 64
svd = TruncatedSVD(
    n_components=N_COMPONENTS,
    n_iter=7,
    random_state=RANDOM_STATE,
)
X_reduced = svd.fit_transform(X_sparse)
X_reduced = Normalizer(copy=False).fit_transform(X_reduced)

print("reduced:", X_reduced.shape)
print("explained variance:", f"{svd.explained_variance_ratio_.sum():.2%}")
```

3. Обучаем один baseline с k=20

Silhouette считаем на фиксированной подвыборке, иначе попарные расстояния будут дорогими. Рост silhouette не гарантирует рост скрытого ARI: метрики отвечают на разные вопросы.

Для точки i величина a_i — среднее расстояние до **других объектов** своего кластера; b_i — наименьшее среднее расстояние до объектов другого кластера.

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}, \qquad -1 \le s_i \le 1.$$

Итоговый silhouette — среднее по выбранным объектам. Это не расстояния до центров. Один кластер не позволяет вычислить silhouette; выбранная метрика здесь — cosine.

```
N_CLUSTERS = 20  # Стартовая гипотеза, не число истинных классов.
rng = np.random.default_rng(RANDOM_STATE)
silhouette_index = np.sort(rng.choice(len(X_reduced), size=1200, replace=False))

model = MiniBatchKMeans(
    n_clusters=N_CLUSTERS,
    batch_size=1024,
    n_init=5,
    random_state=RANDOM_STATE,
)
labels = model.fit_predict(X_reduced)
silhouette = silhouette_score(
    X_reduced[silhouette_index], labels[silhouette_index], metric="cosine",
)
cluster_sizes = pd.Series(labels).value_counts().sort_index()
print("k:", N_CLUSTERS, "cosine silhouette:", round(silhouette, 4))
print("cluster size summary:")
print(cluster_sizes.describe().round(1))
```

4. Интерпретируем диагностику

Для baseline фиксируем `k=20` как прозрачную стартовую гипотезу, а не как «истинное» число классов. Silhouette оценивает геометрию в выбранном пространстве. Размеры кластеров помогают заметить вырожденное решение. Скрытый ARI вычисляет платформа; локально он недоступен.

Проверка устойчивости между seed предлагается как самостоятельный эксперимент после baseline и не входит в обязательные 35 минут. Инструкция находится в конце ноутбука; готовой ячейки для второго запуска здесь нет.

5. Собираем отправку

Номера кластеров произвольны: ARI сравнивает пары объектов, а не буквальные значения меток. Важно сохранить все ID и их исходный порядок.

```
submission = sample[["ID"]].copy()
submission["TARGET"] = labels.astype(int)
submission.to_csv("baseline_submission.csv", index=False)

assert submission.columns.tolist() == ["ID", "TARGET"]
assert submission["ID"].equals(sample["ID"])
assert len(submission) == X_sparse.shape[0]
assert np.isfinite(submission["TARGET"]).all()
assert 2 <= submission["TARGET"].nunique() <= N_CLUSTERS
print("baseline_submission.csv ready:", submission.shape)
print("clusters:", submission["TARGET"].nunique())
print(submission.head())
```

Самостоятельный эксперимент: последние 10 минут практики

0–2 минуты: сохраните исходный `baseline_submission.csv` отдельной копией, запишите исходную метрику и сформулируйте гипотезу.

2–7 минут: добавьте новую кодовую ячейку после baseline и выполните пример ниже. Меняется один параметр. Исходная модель, представление, split/подвыборка и CSV сохраняются. Улучшение не гарантировано: отсутствие заметного эффекта тоже результат.

```python
from sklearn.base import clone

# Представление и подвыборку сохраняем; исходные labels не меняем.
experiment = clone(model).set_params(n_clusters=15)
experiment_labels = experiment.fit_predict(X_reduced)
experiment_silhouette = silhouette_score(
    X_reduced[silhouette_index], experiment_labels[silhouette_index], metric="cosine"
)
print("silhouette k=20:", silhouette)
print("silhouette k=15:", experiment_silhouette)
print(pd.Series(experiment_labels).value_counts().sort_index())
```

7–10 минут: сравните числа и объясните наблюдение двумя предложениями. Что изменилось в модели и чего этот эксперимент ещё не доказывает? Не выбирайте ответ по скрытому leaderboard вместо локального протокола.

Для отдельной проверки устойчивости можно обучить модель с другим seed и сравнить разметки через `adjusted_rand_score`. Такой ARI сравнивает запуски и не использует скрытый эталон. Дополнительные алгоритмы и ассоциативные правила не входят в эти 35 минут; для правил нужны осмысленные транзакции, а не координаты SVD.